ビジュアルノベルエンジンのための解析可能なモデル

プッシュダウン型の物語コアによる
リアルタイムプレビュー・任意ジャンプ・頑健なセーブ

設計提案 · チーム議論用草案 · 2026 年 10 月 22 日

概要：ビジュアルノベル（VN）のデバッグが難しいのは、正しい振る舞いが作者の頭の中にしかないからである。合流後にどの立ち絵が画面に出ているべきか、ある台詞がどのルートに属するか、どのフラグが成り立っていなければならないか。既存のツールは、正しい状態のまま任意の地点へジャンプすることができない。本提案では、VN を**選択履歴上の決定的トランスデューサ**としてモデル化する。有限データ上のプッシュダウン制御構造が物語状態を計算し、純粋関数がその状態を演出に写す。この一つの決定によって、セーブ、プレビュー、ジャンプ、静的検査が、よく研究された理論、すなわちプッシュダウン到達可能性、決定図による記号的モデル検査、抽象解釈、半順序簡約から自然に導かれる。本文書では、モデル、モデルを解析可能に保つためのスクリプト言語の制約、セーブ形式、プレビューの仕組み、リンター、段階的計画を定める。決めるべき論点は 11 章にまとめる。

本草案は、2026 年 10 月 16 日付・10 月 19 日改訂の草案（proposal1）に代わるものである。モデル、目標、理論は変わらない。XML スクリプト、独立したシステムスコープ、`.kirp` レンダーファイルは、三つの関数種別をもつ一つの Rust 風言語に置き換えた（3.4 節）。

目次

1 問題と目標 1
2 中核となる抽象化 2
3 制御モデル：有限データ上のプッシュダウン制御 3
4 状態の関数としての演出 11
5 状態爆発の抑制 13
6 組合せ的な選択：パーティー問題 14
7 ライブプレビュー 15
8 セーブ 16
9 ツール：意図を検査可能にする 17
10 段階的計画 17
11 議論すべき未決事項 18
参考文献 18

1 問題と目標

VN の大部分は時折選択肢を挟む線形のテキストだが、その状態は目に見えない。「屋上のシーンの後に違うキャラクターが画面に出ている」といったバグ報告は、そこに至った経路が**どれか**を知っていなければ再現できず、作者が何を見せる**つもりだったか**を知っていなければ診断できない。現状では、チームメイトも AI エージェントもその知識を持たないため、デバッグは作者自身がゲームをやり直すことに頼っている。

目標	要件
G1 リアルタイムプレビュー	台詞を編集すると、ゲームを再起動せずに現在位置のプレビューが更新される。
G2 任意ジャンプ	任意の台詞を、推測した状態ではなく**到達可能で一貫した**状態で開ける。
G3 頑健なセーブ	セーブはスクリプトの編集後も有効であり、有効でなくなる場合はどこで乖離したかをエンジンが正確に示す。
G4 検査可能な意図	作者の前提が、リンター、チームメイト、エージェントが問い合わせられる機械検査可能な事実になる。

1.1 先行事例

既存の二つのツールは、我々が求めるものの半分ずつと、それぞれの限界を示している。

- **Ren'Py の「warp to line」**は、シーングラフ上で対象の行への経路を探索し、その経路上の `scene`、`show`、`hide` 文だけを再実行する。経路上の Python コードは実行されないため変数が未初期化のままになりうるし、作者は `after_warp` ラベルで状態を手作業で補う。経路探索はゲームロジックを無視するので、選ばれた経路が到達可能とは限らず、調べるのも一本の経路だけである [1]。
- inkle の ink 用エディタである **Inky** は、プレイヤーの選択を保持し、再コンパイルのたびに作者が最後に到達した地点まで早送りする [2]。これはトレース再生であり、状態は正しいが、作者がたまたまクリックして進んだ経路に限られる。

本提案はこの二つを組み合わせる。正しさのためには Inky 流の再生を、作者がまだクリックして到達していない場所へ行くためには、ロジックを考慮した探索（Ren'Py の発想を健全に行うもの）を用いる。

2 中核となる抽象化

2.1 VN は選択履歴上のトランスデューサである

Σ を入力イベント（大部分は選択肢）の集合とし、各イベントは安定した識別子をもつとする。一回のプレイセッションは語 $h \in \Sigma^*$ である。エンジンは次の三つの純粋な部品で定義される。

$$\sigma_h = \mathrm{fold}(\delta, \sigma_0, h) \qquad R = f(\mathrm{pos}, \sigma_h, t)$$

ここで δ は物語遷移関数、σ は物語状態（位置、コールスタック、変数）であり、f は現在位置、状態、ローカル時刻 t を画面 R に写す。設計の残りはすべて、δ を安価かつ全域的に保ち、f に隠れた状態をもたせないことから導かれる。

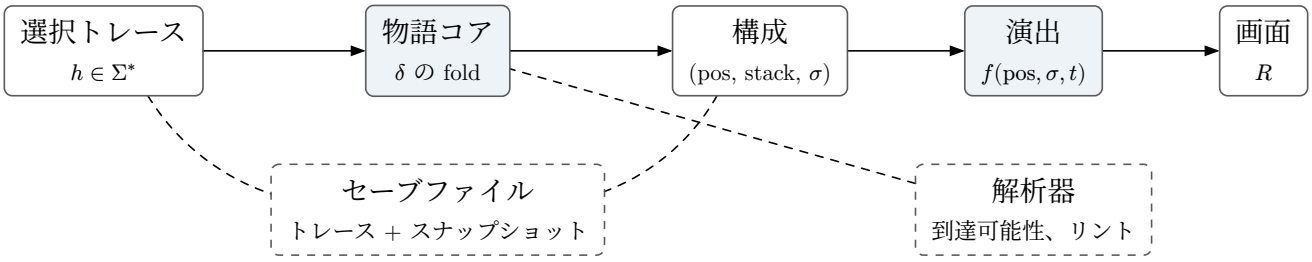


図 1: パイプラインとしてのエンジン。早送り時に再生されるのは物語コアだけであり、演出は目的地で遅延評価される。解析器は物語コアと同じ物語 IR の上で動作する。

2.2 状態はどうあるべきか

これには原理的な答えがある。二つの履歴 h_1, h_2 が**同値**であるとは、将来のどんな入力によっても、画面に現れるものでそれらを区別できないことをいう。

$$h_1 \sim h_2 \iff \forall z \in \Sigma^*.\ \mathrm{screens}(h_1 z) = \mathrm{screens}(h_2 z).$$

これは Myhill–Nerode 合同 [3], [4] を、受理機械から出力を生成する機械へ持ち上げたものである。生の履歴は最大の状態、同値類 $[h]$ は最小の状態であり、作者のフラグはその中間に位置する。本提案の大部分は、エンジンの状態を $[h]$ に近づけること、あるいはそもそもそれを計算する必要をなくすことに関するものである。

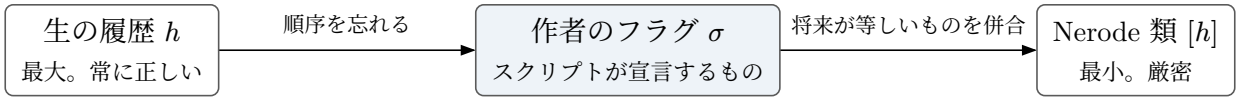


図 2: 状態の三つの粒度。生存解析と双模倣最小化（5.4 節）は中央から右へ向かう。

3 制御モデル：有限データ上のプッシュダウン制御

3.1 停止性問題はスタックではなくデータから生じる

プッシュダウンオートマトンそれ自体の到達可能性は決定可能である。決定不能性は非有界な**データ**から生じる。条件ジャンプをもつ非有界カウンタが二つあれば、スタックがなくても既にチューリング完全な Minsky 機械になる [5]。そこで規則は次のとおりとする。

- **制御**は再帰してよい。シーンは他のシーンを呼び出して戻る。ループも許す（ハブマップ、日付の繰り返し）。
- **データ**は有限の領域を動く。真偽値、列挙型、有界または飽和する整数。

結果は有限制御のプッシュダウンシステムであり、その到達可能性（pre* と post*）はシステムの大きさに対して多項式時間で計算できる [6], [7]。計画上の注意点が一つある。システムの大きさは真偽値変数の個数に対して指数的であり、そのために記号的表現が必要になる（5 章）。

実際に要求するのは「ゲームが停止する」よりも弱い性質であり、上の規則のもとでは次の二つがともに成り立つ。

1. **各ステップが停止する。** 二つの入力地点の間のコードは全域的である。そのためには、すべてのループ**と**すべての呼び出しの循環が入力地点を通る必要がある。入力の前に自分自身を呼ぶシーンは、一ステップの中でスタックを無限に伸ばしてしまう。
2. **到達可能性の問い合わせが決定可能である。** たとえば「412 行目に `MET_B` が偽の状態で到達できるか」。

3.2 プッシュダウン言語とはどういうものか

プッシュダウンシステムをプログラムの形で表したものが**ブールプログラム**（Ball と Rajamani の SLAM プロジェクト [8], [9]）であり、そのグラフ版が**再帰状態機械**（RSM）[10] である。Kagari では、この制約はちょうど一つの関数種別 `pushdown fn` にだけ適用され、物語のコードはすべてこれで書く（3.4 節）。`#[scene]` を付けた `pushdown fn` は**シーン**であり、ステップで中断しうる RSM のコンポーネントである。これを付けない `pushdown fn` は、入力なしで実行される即時手続きである。他の種別である `fn` と `const fn` では言語のすべてを使える。`pushdown fn` の内部では次のとおりである。

pushdown fn で許されるもの	禁止されるもの（とその理由）
有限型のローカル変数、引数、戻り値、ストーリー静的変数（static mut）。有限型とは bool、()、Int<lo, hi>、有限のペイロードをもつ列挙型、有限型からなる構造体・タプル・配列 [T; N]、Option<T>、Text 定数、アセットハンドル	Rust の整数（u8 … i128、usize）、Rat、Decimal、String、Vec、マップ、Screen。これらはカウンタを、したがってチューリング完全性を再び持ち込む
シーン内では、シーン（再帰を含む）、即時手続き、push_step の呼び出し。即時手続き内では、即時手続きの呼び出し（呼び出しグラフは非巡回）。両方で、畳み込みまたは表化される const fn の呼び出し（3.4 節）	fn の呼び出し。チューリング完全なコードは σ を非有界にしてしまう
種別 scene fn、pushdown fn、const fn の関数ポインタを σ に格納して呼び出すこと。名前付き関数の集合は有限なので、ポインタ経由の呼び出しは場合分けであり、proposal1 における有限列挙型上の計算呼び出しにあたる	第一級クロージャ（格納、返却、捕捉されるもの）。クロージャは第二級であり、引数として渡して呼び出すことしかできず、即時的である。つまりクロージャはステップを踏むこともシーンを呼ぶこともできない。高階再帰スキームは決定可能だが、階数 n に対して n-EXPTIME 完全である [11]
シーン内では、本体のすべての経路がステップを通る loop と while、およびステップを通る再帰の循環。どこでも、定数の範囲または配列上の for	ステップも上界もないループや呼び出しの循環。一つのステップが停止しないおそれがある
push_step（入力）。s!(…) と choice!{…} のフォームで書く。出力はステップの render 属性を通じて行う	画面、実時間、OS の状態、メタ（メタ層、8.3 節を参照）の読み取り。これができるのは fn のコードだけであり、pushdown fn はそれを呼べない

表 1: pushdown fn で許されるものと禁止されるもの。各規則は型・種別検査器が検査するので、違反は型エラーになる。

proposal1 にあった規則が一つ、この表から消えている。別のシーンの途中へジャンプすることは、呼び出しと戻りの正しい入れ子を壊すため、検査が必要だった。今ではそれは構成上不可能である。言語には if、match、loop、while、for、break、continue、return はあるが、goto も位置としてのラベルもないので、制御が関数を離れるのは呼び出しか戻りによってだけである。既約でない制御フローは、設計上表現できない。

スタックフレームは戻りアドレスと呼び出し元のローカル変数の組である。どちらも有限なので、スタックアルファベットは有限であり、これはまさにプッシュダウンシステムの定義である。

3.3 スタックが意味をもつとき：学園祭の例

基本的な学園祭のハブにはスタックは**不要**である。プレイヤーは校内マップから模擬店 A、B、C のいずれかへ行き、物語が進み、制御は常に同じ場所、つまりマップへ戻る。再訪した模擬店は短いメッセージを表示して戻る。各模擬店は一か所からしか呼ばれないので、呼び出しはインライン展開できる。記憶は位置と三つの VISITED ビットだけなので、これは有限オートマトンである。

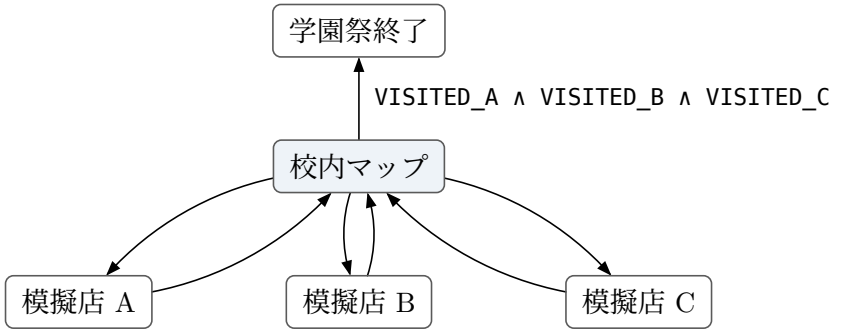


図 3: 一日分のハブ。すべての模擬店が同じノードへ戻るので、一つの選択肢を囲むループで足りる。状態空間は制御ノード 5 個 × フラグ値 2^3 通りである。

スタックが役に立つのは、一つのサブシーンが異なる場所から入られ、異なる場所へ戻らなければならないときである。

- **ハブの再利用：**学園祭が 1 日目（その後は寮へ）と 2 日目（その後はエンディング判定へ）に行われる。
- **割り込み：**どの模擬店の中でもマップ上でも電話がかかってくることがあり、その後ゲームはちょうど元の場所から再開する。

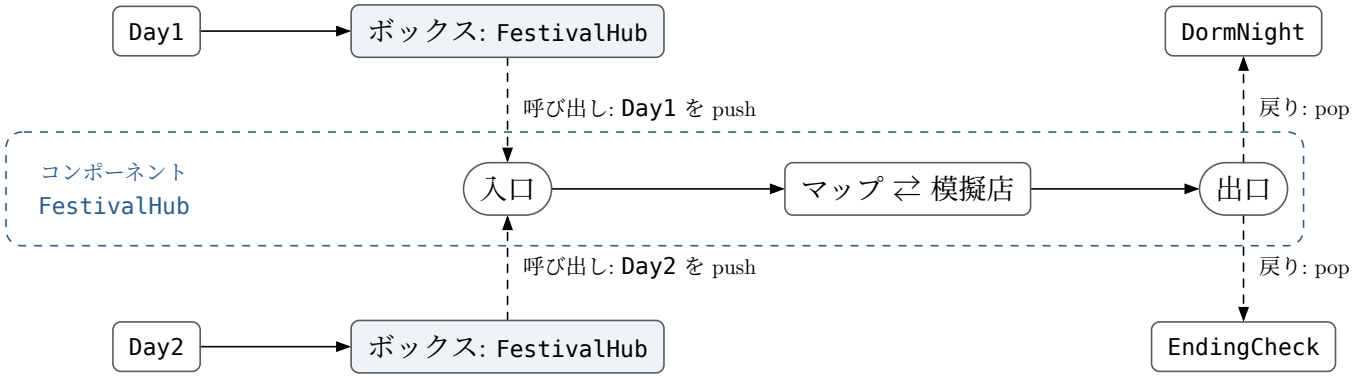


図 4: 再帰状態機械。ボックスは別のコンポーネントへの呼び出しであり、スタックはどのボックスへ戻るかを覚えている。スタックがなければ、`FestivalHub` を呼び出し元ごとに複製しなければならない。

> **決定案**
> プッシュダウンモデルを採用する。その実用上の価値は**簡潔さ**にある。呼び出しの深さが有界なら、受理できるものは有限オートマトンとまったく同じだが、有限オートマトンでは共有されるシーンを呼び出し箇所ごとに一つずつ複製する必要があり、入れ子になれば複製は掛け算で増える。非有界な再帰を許せば、決定可能性を損なうことなく真の文脈自由な表現力が得られる。

Kagari では、この例の 1 日目の部分は次のようになる。構文は 3.4 節で説明する。フォームを使うので、これは `.kix` ファイルである。

```
static mut VISITED_A: bool = false;
static mut VISITED_B: bool = false;
static mut VISITED_C: bool = false;

enum Map { A, B, C, Leave }

#[scene]
#[entry]
```

```
pushdown fn Day1() -> Ending {
    s!(t!["今日から学園祭が始まる。"]);
    FestivalHub();
    DormNight();
    Ending::Normal
}

#[scene]
pushdown fn FestivalHub() {
    loop {
        match choice!{
            id = "map",
            t!["模擬店 A へ行く"] => Map::A,
            t!["模擬店 B へ行く"] => Map::B,
            t!["模擬店 C へ行く"] => Map::C,
            t!["立ち去る"] if VISITED_A && VISITED_B && VISITED_C => Map::Leave,
        } {
            Map::A => StallA(),
            Map::B => StallB(),
            Map::C => StallC(),
            Map::Leave => break,
        }
    }
}

#[scene]
pushdown fn StallA() {
    if VISITED_A {
        s!(t!["模擬店はもう片付けを始めている。"]);
        return;
    }
    VISITED_A = true;
    s!(who = "Hanako", render = hanako_jumps, t!["模擬店 A へようこそ！"]);
}

const fn hanako_jumps(mut screen: Screen) -> Screen {
    screen.entity(HANAKO).cord.apply_animation(jump(20));
    screen
}
```

`loop` はすべての経路で選択肢を通るので、各ステップは停止する。`Leave` のアームは `if` ガードが成り立つときにだけ提示される。行き詰まり選択肢の検査により、常にいずれかのアームが提示されることが証明される。

3.4 一つの言語、三つの関数種別

proposal1 には、XML の物語（`.kis`）、独立した XML のシステムスコープ、Rust 風のレンダーファイル（`.kirp`）、Rust 風の拡張ファイル（`.kix`）があった。スコープが二つ、ファイル種別が三つ、構文が二つである。これらをすべて、三つの関数種別をもつ一つの Rust 風言語で置き換える。

種別	意味	実行時期
`fn`	チューリング完全	実行時：システム（UI、メニュー、セーブ、設定）
`pushdown fn`	有限データ上のプッシュダウン制御。`#[scene]` はステップで中断しうる	実行時、物語の内部
`const fn`	純粋、決定的、チューリング完全。評価される箇所では全域的	コンパイル時（レンダー、定数、表化）。実行時に呼ぶこともできる

表 2: 三つの関数種別。

呼び出し元 / 先	`fn`	`pushdown fn`	`const fn`
`fn`	可	不可。`#[entry]` シーンを通じて物語を**開始**することだけができる	可
`pushdown fn`	不可	可。シーンはシーンと即時手続きを呼び、即時手続きは即時手続きだけを呼ぶ	可（畳み込みまたは表化）
`const fn`	不可	不可	可

表 3: 呼び出し規則。行が呼び出し元、列が呼び出し先である。禁止された呼び出しは型エラーになる。

関数ポインタ型は種別を伴う。`fn(A) -> B`、`const fn(A) -> B`、即時手続きには `pushdown fn(A) -> B`、シーンには `scene fn(A) -> B` を用いる（暫定構文。`scene` は型の中でだけキーワードになる）。したがって `pushdown fn` はポインタを経由しても `fn` に到達できず、即時手続きはシーンに到達できない。const であることは推論されない。`const fn` は明示的に書き、リントがそれを提案することはある。

3.4.1 なぜ制約を関数に置くのか

プッシュダウン制約は、一か所にある一つの性質である。属性でも、ファイルのモードでも、独立したスコープでもない。`pushdown fn` は `pushdown fn` と `const fn` だけを呼べるので、物語の断片は呼び出しについて閉じており、解析器が見る必要があるのは `#[entry]` シーンから到達可能な関数だけである。それ以外は、解析に影響を与えずにチューリング完全であってよい。メニュー、セーブ画面、設定は、第一級クロージャ、シグナル、コレクションを使う普通の `fn` コードである。proposal1 では同じ分離のために、独自の構文をもつ第二のスコープが必要だった。

同じ理屈でクロージャの規則も決まる。クロージャは値で捕捉する。`fn` と `const fn` では第一級であり、UI のハンドラにはそれが必要である。`pushdown fn` では第二級であり、引数として渡して呼び出すことはできるが、格納、返却、捕捉は決してできない。また即時的でもある。クロージャはステップを踏むこともシーンを呼ぶこともできないので、すべてのステップは名前付きシーン内の一点になる。クロージャを格納できると、捕捉された環境が σ に入り、プログラムは高階再帰スキームになる。そのモデル検査は決定可能だが、階数 n に対して n-EXPTIME 完全である [11]。名前付き関数ポインタにはこの問題はない。その値域は名前付き関数の有限集合だからである。

3.4.2 チューリング完全なコードからの唯一の経路

`fn` コードから物語へ入る唯一の方法は、`#[entry]` シーンを開始することである。エントリーシーンは有限な引数型と戻り値型をもつシーンであり、`#[entry]` と `#[scene]` の両方を付けて書く。これらは解析の根であり、システムが開始できる唯一の関数である。戻る方法は、エントリーシーンの戻り値（システムの終了ハンドラがこれを受け取り、網羅的に処理しなければならない）と、書き込み専用のメタ（8.3 節）だけである。したがって物語全体は、その開始から任意のステップまでプッシュダウン

であり、システムが開始できるすべての実行について G2 と G3 が成り立つ。解析器はエントリー引数のすべての値について量化する。

```
// story.kix
static CLEARED_MIO: Meta<bool> = meta(false);

enum Ending { Normal, Mio }
enum Confess { Yes, No }
enum Answer { Accepted, Rejected }

#[scene]
#[entry]
pushdown fn Prologue(cleared_mio: bool) -> Ending {
    if cleared_mio {
        s!(t!["今回は屋上の扉の鍵が開いている。"]);
    }
    let answer = Confession();
    match answer {
        Answer::Accepted => {
            CLEARED_MIO.write(true);
            Ending::Mio
        }
        Answer::Rejected => Ending::Normal,
    }
}

#[scene]
pushdown fn Confession() -> Answer {
    s!(who = "Mio", t!["何か言いたいことがあるの？"]);
    match choice!{
        id = "confess",
        t!["打ち明ける"] => Confess::Yes,
        t!["何も言わない"] => Confess::No,
    } {
        Confess::Yes => Answer::Accepted,
        Confess::No => Answer::Rejected,
    }
}

// system.kis: チューリング完全。std::system の名前は暫定
fn new_game() {
    nav::start(Prologue, (CLEARED_MIO.get(),), |ending| match ending {
        Ending::Normal => nav::goto(Title),
        Ending::Mio => nav::goto(Credits),
    });
}
```

3.4.3 const 関数と表化

`pushdown fn` が `const fn` を呼べるのは、すべての引数が定数式であるか有限型であるときである。引数が定数なら、呼び出しはコンパイル時に畳み込まれる。実行時の有限な引数に対しては、コンパイラはその関数を引数の定義域全体にわたって**表化**する。これにより、実行時に受け取りうるすべての入力で停止することが証明され、関数は有限関係に変わる。解析器はそれを他の有限遷移と同じよう

に扱う。呼び出し全体が定数でない限り、結果は有限型でなければならない。定義域が大きすぎる場合や、評価燃料の範囲内で評価が停止しない場合はコンパイルエラーになる。

```
enum Route { None, Mio, Haruka, Rin }

// 実行時の引数で pushdown コードから呼ばれる：4 × 6 通りの値にわたって表化される。
const fn bonus(route: Route, affection: Int<0, 5>) -> Int<0, 2> {
    match (route, affection) {
        (Route::Mio, 4..=5) => 2,
        (Route::Mio, _) | (_, 5) => 1,
        _ => 0,
    }
}
```

3.4.4 ファイル

拡張子	構文
`.kis`	Rust 風言語
`.kix`	`.kis` にフォームを加えたもの：`s!(…)`、`choice!{…}`、`t![…]`、UI フォーム

拡張子が選ぶのは構文だけであり、意味論は決して変わらない。どちらのファイルにもどの関数種別でも書ける。フォームは `.kix` でだけ使え、`.kis` ファイルではフォームの脱糖先である構造体式を書く。そのため物語は通常 `.kix` で書く。モジュールは Rust 風（`mod`、`pub`、`use`、`::` によるパス）であり、ルートファイル一つ（慣例では `main.kix`）がコンパイラへの入力になる。

3.4.5 物語フォームは糖衣構文である

フォームは `name!(…)`、`name![…]`、`name!{…}` と書く。フォームはマクロではない。マクロは存在しない。フォームは組み込み構文の固定された集合であり、普通の構造体式に、ステップの場合は `push_step` の呼び出しに脱糖される。各フォームの区切り記号はちょうど一種類であり、区切り記号がその形を表す。

区切り記号	形	例
`name!(…)`	属性、次に固定個数の要素	`s!`：`t![…]` は高々一つ
`name![…]`	要素のみ、個数は任意、属性なし	`t![…]`
`name!{…}`	属性、次に任意個数の要素	`choice!{…}`

属性は `key = expr` の形でカンマで区切り、常に要素より前に置く。省略可能な属性は省いてよい。

中断する唯一のプリミティブは `std::narrative` にある `push_step` であり、組み込みの `pushdown fn push_step<S: Step>(s: S) -> S::Output` である。`s!` は普通のステップである。その属性はすべて省略可能で、`who`（話者の表示テキスト。省略すると地の文）、`voice`、`render` があり、唯一の省略可能な要素はテキストである。話者は一ステップにつき一人である。`s!()` はメッセージなしで待つ。このフォームの値は `()` である。

```
s!(who = "Azusa", voice = AZUSA_0072, render = r, t!["ぬっ"]);
// は次と同じ
push_step(Say {
    who: Some("Azusa"),
```

```
    voice: Some(AZUSA_0072),
    render: Some(r),
    text: Some(t!["ぬっ"]),
});
```

`choice!{…}` は一つのステップである。その属性は、関数内で一意な文字列リテラルである `id` と、省略可能な `render` である。要素は `t![…] => V` または `t![…] if cond => V` の形のアームで、カンマで区切る。ガード付きのアームは `cond` が成り立つときにだけ表示される。すべてのバリアントは、作者が宣言したペイロードなしの一つの列挙型から取り、それぞれ高々一回しか現れない。フォームは選ばれたバリアントに評価され、分岐はそれに対する普通の網羅的な `match` で行う。つまりアームはコードブロックではなくデータである。

`t![…]` は `Text` 値（リッチテキスト）を作る。その項目はカンマなしで並べて書き、そのまま連結される。

- Rust のエスケープをもつ文字列リテラル。テキストはその内容そのものであり、改行は `\n` である。
- ルビ：グループルビは `["数多"]("あまた")`、モノルビは `["漢字"]("かん"|"じ")` と書く。モノルビでは読みの部分の数が親文字の数と等しい。
- テキスト表現値：`!["赤い"](color_text(RED))`。対象は `TextEffect` 型（暫定名）の定数式であり、コンパイル時に畳み込まれる。これは関数ではなく値なので、テキストの演出も画面と同じように解析可能なままである。強調は `!["you"](emphasis())` と書く。

```
s!(t!["これは" ["数多"]("あまた") "の" !["赤い"](color_text(RED)) "花"]);
```

`#[scene]` は `pushdown fn` を**シーン**として印付ける。シーンはモデル検査の単位であり、一つの RSM コンポーネントである。中断するために不純である `push_step` を呼べるのはシーンだけであり、他のシーンを呼べるのもシーンだけである。シーンは普通の呼び出し構文で呼ぶ（`FestivalHub();`、`let answer = Confession();`）。シーンどうしは `.await` なしに直接呼び合う。これは、他の中断関数からしか呼べない Kotlin の `suspend` 関数と同様である [12]。ステップは入力があるまで物語全体を中断する。複数の終わり方がありうるシーンは列挙型を返し、呼び出し元はそれに対して `match` する。3.1 節の停止性検査はシーンに適用される。

`#[scene]` を付けない `pushdown fn` は**即時手続き**である。即時手続きは有限型をもち、ストーリー変数を読み書きでき、他の即時手続きと `const fn` を呼べる。`push_step` もシーンも呼べず、再帰できず（即時手続き間の呼び出しグラフは非巡回）、有界なループ（定数の範囲または配列上の `for`。`loop` と `while` は不可）しかもたない。したがって構成上、入力なしで停止し、解析器はその呼び出しを、呼び出し元シーンの中の一つの即時遷移として扱う。

3.4.6 ビューとしてのノードエディタ

RSM は文字どおりノードグラフである。各**コンポーネント**は入口ノードと出口ノードをもつグラフであり、**ボックス**は、出口が呼び出し元に結線された呼び出しを表す [10]。Kagari では、コンポーネントは `#[scene]` 関数、ボックスは `#[scene]` 関数の呼び出し、辺はその構造化された制御フロー、`s!` と `choice!` のステップは入力ノードである。シーンをグラフとして表示するノードエディタは引き続き作れるが、もはや XML は往復変換の基準となる正準形ではない。テキストが正準形であり、テキストとして差分を取りマージできることに変わりはない。

> **決定案**
> Kagari は三つの関数種別をもつ一つの Rust 風言語とする。`fn`（チューリング完全、システム）、

> pushdown fn（物語）、const fn（レンダー、定数、表化される補助関数）である。プッシュダウン制約は pushdown fn の性質であってそれ以外の何物でもなく、型・種別検査器が検査する。その中で #[scene] はモデル検査の単位を印付ける。ステップを踏みシーンを呼べるのはシーンだけであり、他の pushdown fn は即時手続きである。fn コードから物語への唯一の経路は、有限の引数で #[entry] シーンを開始することであり、戻りの経路はその戻り値と書き込み専用のメタだけである。物語フォーム（s!、choice!、t!）は、構造体式と push_step のための固定された組み込みの糖衣構文であり、.kix で使える。.kis と .kix のテキストがバージョン管理上の正準形であり、ノードエディタを作る場合はそのビューとする。

4 状態の関数としての演出

4.1 φ 問題はどこから来るか

show bg roof のような命令的なコマンドは、隠れた演出変数に代入する。二つの分岐が合流する地点では、SSA 構築 [13] とまったく同じように、それらの変数に φ ノードが必要になる。φ を暗黙のままにすると、画面は σ ではなく**経路**に依存し、$f(\text{pos}, \sigma)$ はもはや well-defined ではなくなる。

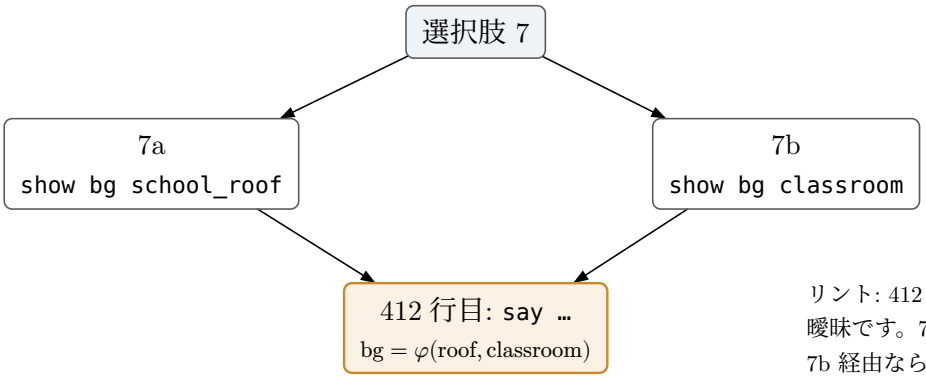


図 5: 合流点における暗黙の φ。修正方法は、合流後にキーフレームを置くか、明示的な状態変数を用いるかのいずれかである。

4.2 併用する三つの仕組み

1. **レンダー関数としてのキーフレームと差分：**各ステップはレンダーをもつことができる。レンダーは render 属性として渡される const fn(Screen) -> Screen である。レンダーの唯一の入力は画面であり、しかも const fn なので、コンパイラがそれを評価し、各レンダーはコンパイル時に完全に判明する。出力が入力に依存しないレンダーは、ビューがモデルの関数である Elm アーキテクチャ [14] と同様に、画面全体を宣言する。これが**キーフレーム**であり、典型的には Screen::empty() から始まる。コンパイラはキーフレームを検出する。それ以外のレンダーは**差分**である。動画コーデックに喩えれば、キーフレームは I フレーム、差分は P フレームであり、シークとは直前の I フレームへ飛んで差分を適用することである。すべての変更はそのアニメーションを指定する。change(..) は使用しなければならない保留中の変更を返し、apply_animation がそれを適用する。アニメーションは画面状態の一部ではなく、再生時には省略される。
2. **σ に昇格させた明示的な φ：**合流後の画面が本当に分岐に依存するなら、その依存は物語状態であり、作者が宣言するかエンジンが具体化するかして、σ に置かなければならない。生存解析によってこのコストは小さく保たれる。その変数は次のキーフレームで死ぬからである。
3. **リントとしての曖昧性：**スクリプトに対する抽象解釈 [15] により、各ステップについて、ありうる演出状態の集合（冪集合束、join は和集合）を、エンティティとコンポーネントごとに、シーンご

とに一つの要約をもって計算する。単集合なら問題なく、それより大きければ、原因となる経路とともに報告する。画面を比較するには、すべてのコンポーネント型が決定可能な等価性をもつ必要がある。コンポーネントは構造的な `Eq` をもつ構造体であり、チューリング完全なコードには厳密な数（`Rat`、`Decimal`）はあっても浮動小数点数はないので、等価性は決定可能なままである。有限性のためには、画面の集合が有界にとどまる必要がある。コンポーネントを読み取ってそこから計算するレンダー（右へ 10 単位動かすなど）は、ループを通るたびに集合を大きくしうる。レンダーはチューリング完全な `const fn` なので、有界性は一般には決定不能である。そこで規則は上限付きの評価とする（暫定）。コンパイラは各ステップに到達する画面上でレンダーを評価し、不動点計算の途中であるステップの集合が固定の上限を超えて大きくなったら、それを大きくしたレンダーをエラーとする。受理されたプログラムはちょうど計算された集合をもつので、これは健全であるが、有界なプログラムの一部を拒否することがある。ループ内で有理数の分母が大きくなる場合も同じ方法で検出される。

> **決定案**
> 演出は、エンティティ・コンポーネント・システム（ECS）型の画面上のレンダー関数 `const fn(Screen) -> Screen` として記述する。エンティティはレンダーが管理し、コンポーネントは構造的な `Eq` をもつ構造体であり、エンティティをアニメーションさせるシステムはエンジン内にある。レンダーは構成上 σ を見ることができない。`render` 属性は関数ポインタ、またはそれに型強制される捕捉なしのクロージャであり、`pushdown fn` 内のクロージャは第二級なので、どのストーリー変数もレンダーに届かない。演出が物語に依存しなければならないときは、スクリプトが `if` や `match` で分岐し、各アームのステップに異なるレンダーを渡す。物語コアは画面を決して読まない。`Screen` は有限型ではないので、`pushdown fn` はそれを保持できない。レンダーはチューリング完全なので、画面の有界性は上限付きの評価によって強制する（暫定）。

```
static mut AFFECTION_MIO: Int<0, 5> = 0;

#[scene]
pushdown fn Roof() {
    s!(render = roof_evening, t!["屋上の風は冷たい。"]);
    match AFFECTION_MIO {
        3..=5 => s!(who = "Mio", render = |mut screen| {
            screen.entity(MIO).appearance.pose
                .change(MIO::SMILE)
                .apply_animation(fade(10));
            screen
        }, t!["来てくれたんだ。"]),
        _ => s!(who = "Mio", render = |mut screen| {
            screen.entity(MIO).appearance.pose
                .change(MIO::NEUTRAL)
                .apply_animation(fade(10));
            screen
        }, t!["……ああ、あなたか。"]),
    }
}

const fn roof_evening(_: Screen) -> Screen {
    let mut screen = Screen::empty();
    screen.background().change(BG::ROOF_EVENING).apply_animation(fade(30));
```

```
    screen.entity(HARUKA).appearance.pose
        .change(HARUKA::SMILE)
        .apply_animation(instant_switch());
    screen
}
```

最初のレンダーは入力画面を無視するのでキーフレームである。`match` のアーム内のレンダーは差分であり、捕捉なしのクロージャとしてインラインで書かれている。

曖昧性リントは、おそらく最も価値の高いデバッグ機能である。これは、現状では「作者しか知らない」種類のバグ、すなわち別ルートから残った立ち絵や、合流後も鳴り続ける BGM をまさに捕まえる。

5 状態爆発の抑制

互いに独立な三択が 20 個あれば $3^{20} \approx 3.5 \times 10^{9}$ 個の状態になる。君と彼女と彼女の恋。[16] のような商業作品の分岐はこの規模だと見積もっている。この数が問題になるのは状態を列挙する場合だけであり、我々は決して列挙しない。

5.1 セーブとジャンプは状態ではなくトレースである

行 X を表示するのに「X における状態」は要らない。必要なのは X に到達する**何らかの**履歴と、その再生である。演出なしで δ を再生するのは安価であり（描画もアニメーションもアセット読み込みもない）、f は目的地で一度だけ評価される。

5.2 状態の集合は記号的に表す

ある行に到達しうる状態の集合は、既約順序付き二分決定図 [17]、あるいは列挙型フラグには多値決定図 [18] で表す。互いに独立な三択 20 個は、およそ 20 ノードの決定図になる。爆発が起きるのはフラグが本当に複雑に絡み合う場合だけであり、そうした絡み合いはいずれにせよ作者が把握する必要のあるものである。エディタの各行には、その**到達条件**、たとえば $(\mathrm{met}_A \wedge \neg \mathrm{met}_B) \vee \mathrm{route} = C$ を表示できる。

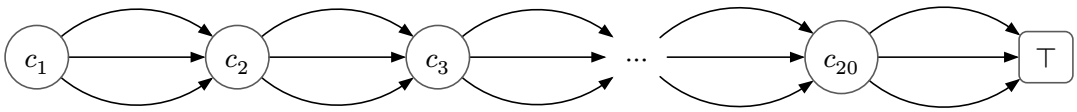


図 6: 互いに独立な三択 20 個の多値決定図。ノード 21 個と辺 60 本で済み、明示的な木なら 3^{20} 枚の葉が必要になる。

5.3 ジャンプは証拠の生成である

「ミオの好感度が 3 以上の場合の行 X を見せて」はモデル検査の問い合わせである [19]。決定図（あるいは SAT ソルバ）は、充足する割り当てと、それを実現する経路を返し、その経路を再生する。到達不能な行はデッドコードであり、報告される。

5.4 データフロー解析による状態の縮小

- **生存解析：**フラグは最後に読まれた後は死んでいるので、その地点以降の到達集合の計算から射影して取り除く。これらの解析の手続き間版はグラフ到達可能性問題である [20]。
- **双模倣最小化：**将来が同一の状態どうしを分割洗練によって併合する [21]。これは Nerode 商の計算可能な近似である。実際には、ルート確定後には序盤のフラグの大半がつぶれる。

5.5 遅延的・インクリメンタルな評価

上記のどれも事前には計算しない。各解析は「ノード N における到達集合」や「σ のもとでの N における演出」のようなメモ化された要求駆動の問い合わせであり、自己調整計算 [22] や Adapton [23]

の流儀に従う。その実用実装として Salsa [24] がある。一つのシーンを編集すると、それに依存する問い合わせだけが無効化される。これが、編集中のリアルタイムプレビューを現実的なコストに収める鍵である。

6 組合せ的な選択：パーティー問題

パーティーでプレイヤーは n 人の客それぞれと、任意の順序で一度ずつ話す。完全な順序は $n!$ 通り、順序付きの接頭辞は $\lfloor e \cdot n! \rfloor$ 通りある。将来が**誰を**訪ねたかだけに依存するなら、Nerode 状態は訪問済みの部分集合であり、状態数は 2^n になる。これは巡回セールスマン問題に対する Held–Karp アルゴリズムが用いるのと同じ圧縮である [25]。

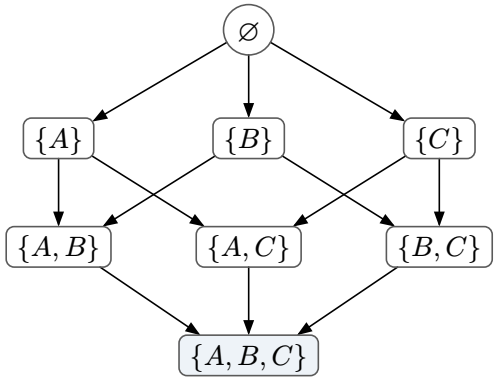


図 7: $n = 3$ の部分集合束。各経路が一つの順序に対応し、状態は共有される。

n	順序付き接頭辞	部分集合 2^n	個数 $n+1$
3	16	8	4
5	326	32	6
8	109 601	256	9
10	9 864 101	1 024	11

表 4: 将来が何に依存するかについての三つの仮定のもとで必要な状態数。

さらなる縮小を、強さの順に挙げる。

- **独立性：**会話 A と B が σ の互いに素な部分にしか触れないなら、AB と BA は同じ状態に到達する。これは Mazurkiewicz トレースの理論 [26], [27] であり、半順序簡約は同値類ごとに一つの代表だけを探索する [28], [29], [30]。
- **対称性：**訪問済みの客の**人数**だけが問題なら、状態は $n+1$ 個である。
- **記号的には、**「n 個の独立なビットのすべての部分集合」という集合は定数「真」の決定図である。

6.1 言語におけるシャッフル演算子

重要な一手は解析器ではなく言語の側にある。有限オートマトンやプッシュダウンオートマトンで「各客にちょうど一度ずつ」を表現するには、2^n 個の制御状態を書き下すしかない。**シャッフル**（インターリーブ）演算子は同じ言語クラスにとどまる（正規言語のシャッフルは正規である）が、指数的に簡潔である [31]。以下はスケッチであり、`each_once` はまだ言語に含まれていない。`loop { match choice!{…} {…} }` のように読めるが、各アームが高々一度しか提示されない点が異なる。

```
enum Guest { Haruka, Mio, Rin, Sensei, Leave }

#[scene]
pushdown fn Party() {
    // スケッチ：`each_once` と `visited` はまだ言語に含まれていない。
    each_once choice!{
        id = "guest",
        t!["ハルカと話す"] => Guest::Haruka,
        t!["ミオと話す"] => Guest::Mio,
        t!["リンと話す"] => Guest::Rin,
        t!["先生と話す"] => Guest::Sensei,
```

```
        t!["パーティーを抜ける"] => Guest::Leave,
    } {
        Guest::Haruka => TalkHaruka(),
        Guest::Mio => TalkMio(),
        Guest::Rin => TalkRin(),
        Guest::Sensei => TalkSensei(),
        Guest::Leave => break,
    }
}
```

文脈自由言語のシャッフルは一般には文脈自由ではない [32]。そのため各分岐は**アトミック**に実行しなければならない。つまり会話は、次の会話が始まる前に戻る呼び出しである。そうすれば順序依存性は、発見されるものではなく宣言されるものになる。

```
#[scene]
pushdown fn TalkMio() {
    if visited(Guest::Haruka) {  // 訪問済み集合への依存の宣言
        s!(who = "Mio", t!["来るって、ハルカから聞いてたよ。"]);
    }
    // …
}
```

解析器は状態を、ビット集合と宣言された依存関係として読む。順序が可換であることを知るために順序を探索することは決してない。

7 ライブプレビュー

> **決定案**
> スナップショットは選択肢の地点で取り、画面を含める。プレビューは、直前の選択肢スナップショットを復元し、物語コアをカーソルまで再実行する。その途中ではレンダーをアニメーションなしで実行する。調べるべき行が**どれか**を突き止めるのは作者に任せ、リンターがそれを支援する。証拠に基づくジャンプ（5.3 節）は後のフェーズとする。

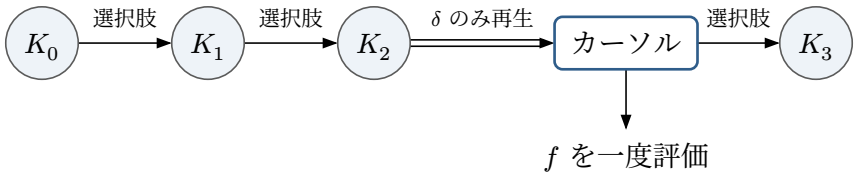


図 8: 編集後のプレビュー。スナップショット K_2 を復元し、物語コアをカーソルまで再生して、一度だけ描画する。

詳細を三点挙げる。

- **スナップショットはコールスタックを含む：**選択肢の後、`return` はどこへ戻るかを知っていなければならない。ハブの中ならフレームは一つだが、2 日目の模擬店 A での電話の最中なら三つになる。
- **スナップショットなしのロードとジャンプ：**物語コアは最初から再生するが、レンダーは経路上の最後のキーフレーム以降だけで実行する。
- **ホットリロード：**作者が関数を編集すると、現在のトレース上で最初に編集されたステップより後のスナップショットが無効化され、再生によって再構築される。これは Inky が再コンパイル後に

行うのと同じである [2]。記録された選択肢がもはや存在しない場合、プレビューはその選択肢で停止し、そこを強調表示する。

8 セーブ

8.1 形式

部分	内容	役割
トレース	エントリーシーンのパスとその引数。安定した選択肢 ID の列。最後の選択肢以降のステップ数。スクリプトのバージョンハッシュと PRNG シード	正準
スナップショット	位置、コールスタック、σ（関数ポインタはパスで格納）、昇格させた φ 変数、PRNG 状態、画面	キャッシュ

選択肢 ID は、それを囲む関数のパス、選択肢の `id`、選ばれたバリアントの名前からなる。たとえば `story::Confession/confess/Confess::Yes` である。呼び出し文脈は再生によって与えられるので、選択肢の `id` は関数内で一意でありさえすればよい。ID は決して行番号ではない。関数、選択肢の `id`、バリアントの名前を変更すると、ロード時の乖離となり、その ID とともに報告される。こうしてセーブは到達可能性の証拠となる。格納された引数でエントリーシーンからトレースを再生すると、セーブされた位置に到達する。乱数には状態が σ の一部である決定的な PRNG を用いるので、再生は厳密である。

8.2 スクリプトのバージョンをまたいだロード

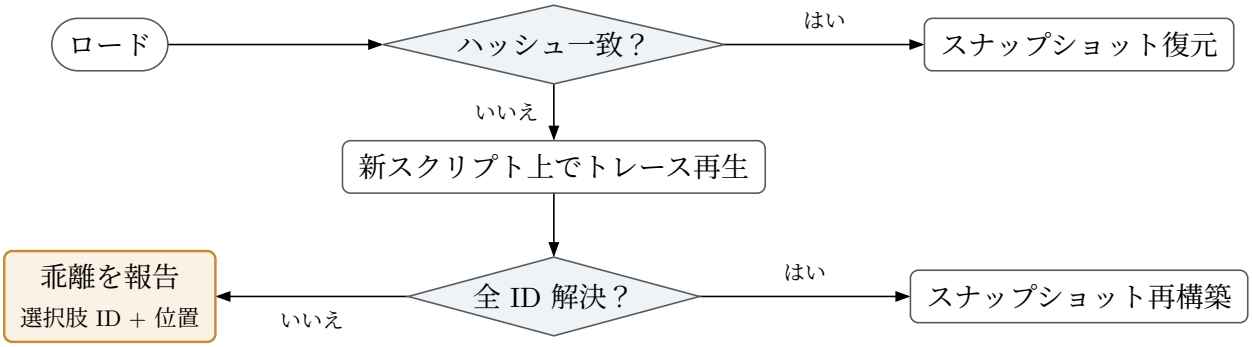


図 9: セーブのロード。スナップショットはスクリプトが変更されていないときにだけ信頼する。

8.3 メタ層

周回をまたいで永続する状態（解放要素、既読フラグ、ギャラリー）は独立した外側の層に置き、`static CLEARED_MIO: Meta<bool> = meta(false);` のような `Meta<T>` ハンドルとして宣言する。この層を意図的に物語の一部にする作品もあり、君と彼女と彼女の恋。はその有名な例である [16]。物語がメタを読むのはエントリー引数を通じてだけである。`Meta<T>` の読み取りは `fn` 専用なので、システムがそれを読み、`#[entry]` シーンを開始するときに値を渡す（3.4 節）。セーブは引数をトレースとともに格納し、解析器は引数のすべての値について量化するので、それらを普通の有限変数として扱え、デバッガでは作者が直接設定できる。物語はメタに `meta.write(v)` で書き込み専用で書く。こうした書き込みはロードや巻き戻しで取り消されず、物語がその実行の中でそれを観測することはない。

9 ツール：意図を検査可能にする

9.1 注釈を必要としないリント

リント	検出するもの	手法
到達不能ノード	デッドな行、ありえないルート条件	プッシュダウン到達可能性
曖昧な演出	合流後に残った立ち絵・BGM・背景	抽象解釈
書くだけで読まれない	何にも影響しないフラグ	生存解析
非有界なステップ	入力も上界もないループや呼び出しの循環	手続き間不動点
非有界な画面	読み取り・変更・書き込みによってステップの画面集合を上限を超えて大きくするレンダー	不動点までの const 評価
行き詰まり選択肢	どのアームも提示されない状態で到達しうる選択肢	プッシュダウン到達可能性
非アトミックなシャッフル	途中で割り込まれうる `each_once` の分岐	呼び出し構造の検査

種別の違反はリントではない。`fn` を呼んだり、`Vec` を保持したり、クロージャを格納したりする `pushdown fn` は、どの解析よりも前に型・種別検査器によって拒否される。

9.2 契約

作者はステップやシーンに表明を付けられる。「ここではハルカが画面に出ている」「このシーンは `MET_B` を要求する」「告白の後、`ROUTE` は二度と変わらない」。これらは時相論理の性質 [33] であり、到達可能なすべての状態について検査される。失敗した契約には反例トレースが付き、それはそのままプレビューアで開ける。

9.3 なぜこれが AI エージェントの助けになるのか

現在エージェントが失敗するのは、作者の意図が検査可能な形で書き留められていないからである。このモデルがあれば、エージェントは推測する代わりに、到達条件を問い合わせ、契約を読み、リントを実行し、反例を再生し、足りない契約を提案できる（「412 行目には二種類の背景で到達します。キーフレームを置くべきですか？」）。

10 段階的計画

各フェーズはそれ単独で使え、後のフェーズを必要としない。

フェーズ	成果物	実現される目標
0	`.kis` と `.kix` のパーサ。型・種別検査器。物語 IR（RSM）と物語インタプリタ。スナップショットキャッシュ付きのトレースセーブ	G3
1	ECS 画面上でのレンダーの const 評価。キーフレーム検出。画面を含む選択肢地点のスナップショット。ホットリロードプレビュー	G1
2	小規模スクリプト向けの明示状態解析器。到達不能、曖昧性、生存解析のリント	G4（部分的）

3	記号的到達可能性（MDD）。行ごとの到達条件。証拠ジャンプ。反例付きの契約	G2、G4
4	ライブ解析のためのインクリメンタル問い合わせエンジン。依存関係を宣言する `each_once`。半順序簡約	スケール

フェーズ 2 はあえて明示状態探索から始める。単純であり、IR の妥当性を確かめられ、小規模なプロジェクトなら十分に速い。フェーズ 3 では、リントのインターフェースを変えずにバックエンドを置き換える。

11 議論すべき未決事項

未決事項

1. **整数の範囲：**`Int<lo, hi>` は `+=` と `-=` で飽和し、上界・下界に達することを解析器が証明した場合は警告を出す。好感度のようなカウンタに既定の範囲（たとえば prelude の別名）を与えるべきか。与えるならどの範囲か。
2. **再帰の深さ：**非有界な再帰（完全なプッシュダウン）を許すか、スタックの深さに上限を設けるか（有限オートマトン、解析がより単純）。非有界な再帰を必要とする現在のユースケースはない。
3. **時間制限付きの入力：**オート送り、時間制限付きの選択肢、QTE は Σ のイベントか。一案として、出力が有限のイベント列挙型（たとえば選ばれたアームまたはタイムアウト）であるステップとし、選択肢と同様にトレースに記録する。
4. **φ の方針：**曖昧性リントが発火したとき、エディタはキーフレームの挿入を提案すべきか、状態変数の昇格を提案すべきか、その両方か。
5. **メタ層の範囲：**現在、物語はメタをエントリー引数を通じてだけ読み、書き込み専用で書く。メタ層を物語の一部にする作品にとって、これで十分か。
6. **画面の有界性の上限：**上限をいくつにするか。ステップごとに数えるかシーンごとに数えるか。有界なプログラムの一部が拒否されることは実用上許容できるか。
7. **チューリング完全なコードにおける数：**Rust の整数、`Rat`、`Decimal` のリテラル構文。オーバーフローの挙動（暫定：const 評価中のオーバーフローはコンパイルエラー。実行時は未定）。`Decimal` の除算の精度。
8. **ユーザー定義のトレイトとジェネリクス：**現在存在するのは標準トレイト（`Eq`、`Step`）と、`Int<lo, hi>` と標準型のための const ジェネリクスだけである。作者が独自のトレイトやジェネリクスを必要とするか。必要なら、種別検査や表化とどう相互作用するか。
9. **テキスト補間：**`t![…]` 内の `Text` 型または `Int` 型の `{expr}`（暫定）。補間されたテキストをバックログはどう記録するか。

参考文献

[1] Ren'Py contributors, 「Ren'Py Documentation: Developer Tools (Warping to a Line)」. [オンライン]. 入手先: https://github.com/renpy/renpy/blob/master/sphinx/source/developer_tools.rst

[2] inkle, 「Inky: An Editor for ink, inkle's Markup Language for Writing Interactive Narrative」. [オンライン]. 入手先: https://github.com/inkle/inky

[3] A. Nerode, 「Linear Automaton Transformations」, *Proceedings of the American Mathematical Society*, 巻 9, 号 4, ページ 541–544, 1958.

[4] J. E. Hopcroft, R. Motwani, と J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3 番目 版. Addison-Wesley, 2006.

[5] M. L. Minsky, *Computation: Finite and Infinite Machines*. Prentice-Hall, 1967.

[6] A. Bouajjani, J. Esparza, と O. Maler,「Reachability Analysis of Pushdown Automata: Application to Model-Checking」, *CONCUR '97: Concurrency Theory*, LNCS, vol. 1243. Springer, 1997, ページ 135–150.

[7] S. Schwoon, 「Model-Checking Pushdown Systems」, Doctoral dissertation, 2002.

[8] T. Ball と S. K. Rajamani, 「Boolean Programs: A Model and Process for Software Analysis」, technical report MSR-TR-2000-14, 2000.

[9] T. Ball と S. K. Rajamani, 「Bebop: A Symbolic Model Checker for Boolean Programs」, *SPIN Model Checking and Software Verification*, LNCS, vol. 1885. Springer, 2000, ページ 113–130.

[10] R. Alur, M. Benedikt, K. Etessami, P. Godefroid, T. Reps, と M. Yannakakis, 「Analysis of Recursive State Machines」, *ACM Transactions on Programming Languages and Systems*, 巻 27, 号 4, ページ 786–818, 2005.

[11] C.-H. L. Ong, 「On Model-Checking Trees Generated by Higher-Order Recursion Schemes」, *21st Annual IEEE Symposium on Logic in Computer Science (LICS'06)*, IEEE Computer Society, 2006, ページ 81–90. doi: 10.1109/LICS.2006.38.

[12] JetBrains,「Kotlin Documentation: Coroutines Basics (Suspending Functions)」. [オンライン]. 入手先: https://kotlinlang.org/docs/coroutines-basics.html

[13] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, と F. K. Zadeck, 「Efficiently Computing Static Single Assignment Form and the Control Dependence Graph」, *ACM Transactions on Programming Languages and Systems*, 巻 13, 号 4, ページ 451–490, 1991.

[14] E. Czaplicki と S. Chong,「Asynchronous Functional Reactive Programming for GUIs」, *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, ページ 411–422.

[15] P. Cousot と R. Cousot, 「Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints」, *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, 1977, ページ 238–252.

[16] Nitroplus, 「君と彼女と彼女の恋。 (Totono)」. 2013 年.

[17] R. E. Bryant,「Graph-Based Algorithms for Boolean Function Manipulation」, *IEEE Transactions on Computers*, 巻 C-35, 号 8, ページ 677–691, 1986.

[18] A. Srinivasan, T. Kam, S. Malik, と R. K. Brayton, 「Algorithms for Discrete Function Manipulation」, *IEEE International Conference on Computer-Aided Design (ICCAD)*, 1990, ページ 92–95.

[19] C. Baier と J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[20] T. Reps, S. Horwitz, と M. Sagiv, 「Precise Interprocedural Dataflow Analysis via Graph Reachability」, *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1995, ページ 49–61.

[21] R. Paige と R. E. Tarjan,「Three Partition Refinement Algorithms」, *SIAM Journal on Computing*, 巻 16, 号 6, ページ 973–989, 1987.

[22] U. A. Acar, 「Self-Adjusting Computation」, Doctoral dissertation, 2005.

[23] M. A. Hammer, K. Y. Phang, M. Hicks, と J. S. Foster, 「Adapton: Composable, Demand-Driven Incremental Computation」, *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, ページ 156–166.

[24] Salsa contributors,「Salsa: A Generic Framework for On-Demand, Incrementalized Computation」. [オンライン]. 入手先: https://github.com/salsa-rs/salsa

[25] M. Held と R. M. Karp,「A Dynamic Programming Approach to Sequencing Problems」, *Journal of the Society for Industrial and Applied Mathematics*, 巻 10, 号 1, ページ 196–210, 1962.

[26] A. Mazurkiewicz, 「Concurrent Program Schemes and Their Interpretations」, DAIMI Report PB-78, 1977.

[27] V. Diekert と G. Rozenberg, 編, *The Book of Traces*. World Scientific, 1995.

[28] A. Valmari, 「Stubborn Sets for Reduced State Space Generation」, *Advances in Petri Nets 1990*, LNCS, vol. 483. Springer, 1991, ページ 491–515.

[29] D. Peled, 「All from One, One for All: On Model Checking Using Representatives」, *Computer Aided Verification (CAV '93)*, LNCS, vol. 697. Springer, 1993, ページ 409–423.

[30] P. Godefroid, *Partial-Order Methods for the Verification of Concurrent Systems: An Approach to the State-Explosion Problem*, 巻 1032. LNCS, vol. 1032. Springer, 1996.

[31] A. J. Mayer と L. J. Stockmeyer, 「The Complexity of Word Problems — This Time with Interleaving」, *Information and Computation*, 巻 115, 号 2, ページ 293–311, 1994.

[32] J. Gischer,「Shuffle Languages, Petri Nets, and Context-Sensitive Grammars」, *Communications of the ACM*, 巻 24, 号 9, ページ 597–605, 1981.

[33] A. Pnueli, 「The Temporal Logic of Programs」, *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, ページ 46–57.